
Dubai DB Checklist Documentation

Purpose

The checklist script validates a Dubai DB output file and creates an Excel report. It keeps the existing column-analysis report and adds schema validation so DB1 and DB2 output files can be checked against their respective schema workbooks.

Main script

File:

D:\Dubai Data\CheckList\db_checklist.py

Primary function:

analyze_excel_file(input_file, output_file=None, schema_file=None)

Supported input files

The script supports both CSV and Excel files.

CSV input:

Files ending with .csv are read using pandas read_csv.

Excel input:

Files ending with .xlsx or .xls are read using pandas read_excel from the first sheet.

Unsupported input:

Any other file extension raises an unsupported file type error.

Schema files

DB1 schema:

D:\Dubai Data\CheckList\DB1_Schema.xlsx

DB2 schema:

D:\Dubai Data\CheckList\DB2_Schema.xlsx

DB1 schema columns used:

DB1(Transaction DB)-Actual
Data_Type

DB2 schema columns used:

DB2(Project_DB)-Actual
Data_Type

Automatic schema selection

The checklist detects which schema to use from the uploaded file name.

DB1 detection:

If the input filename contains DB1, the script validates against DB1_Schema.xlsx.

DB2 detection:

If the input filename contains DB2, the script validates against DB2_Schema.xlsx.

Manual schema selection:

A `schema_file` path can be passed to `analyze_excel_file` when a specific schema should be used regardless of automatic detection.

Column presence validation

For every expected column listed in the schema `actual-column` field, the script checks whether the same column exists in the uploaded file.

Pass:

The expected schema column exists in the uploaded file.

Fail:

The expected schema column is missing from the uploaded file.

Extra:

The uploaded file contains a column that does not exist in the schema.

Data type validation

The script validates each present uploaded column against the schema `Data_Type`.

INTEGER:

Passes when the pandas dtype is integer or the non-null values are numeric whole numbers. For example, 10.0 can pass as integer-like when every value is whole-number compatible.

DECIMAL:

Passes when all non-null values can be converted to numeric decimal values.

VARCHAR and TEXT:

Pass when the pandas dtype is object or string.

DATE:

Passes when the pandas dtype is datetime or at least 90 percent of non-null values can be parsed as dates. The script tries both normal date parsing and day-first parsing.

Unknown:

If a column is present but all values are null, the data type result is marked Unknown because there is no value available to prove or disprove the expected data type.

Unsupported schema type:

If the schema contains a `Data_Type` outside the supported types, the data type result is marked Unknown with an issue message.

Generated Excel report

The output report is an Excel workbook. If `output_file` is not supplied, the script creates the report name automatically using the input file stem plus `_analysis_report.xlsx`.

Example:

Dubai_DB1.csv becomes Dubai_DB1_analysis_report.xlsx.

Report sheet: Column Analysis

This sheet contains one row per uploaded column.

Columns included:

- Column Name
- Data Type
- Total Values
- Non-Null Count
- Null Count
- Null Percentage
- Unique Values
- Sample Values (first 3)

Logic:

The script calculates completeness, uniqueness, pandas dtype, and sample values for every uploaded column.

Report sheet: Summary

This sheet contains high-level metrics for the uploaded file and schema validation.

Metrics included:

- Total Rows
- Total Columns
- Total Memory Usage (MB)
- Columns with Null Values
- Columns with 100% Null Values
- Numeric Columns
- Text/Object Columns
- Date/Time Columns
- Schema Expected Columns
- Schema Missing Columns
- Schema Extra Uploaded Columns
- Schema Data Type Failures

Report sheet: First 10 Rows Sample

This sheet contains the first 10 rows from the uploaded file with an added Row_Number column.

Purpose:

It helps quickly inspect whether the file loaded correctly and whether column values look reasonable.

Report sheet: Schema Validation

This sheet contains the DB1 or DB2 schema validation output.

Columns included:

- Validation Type
- Schema
- Schema Column
- Expected Data Type
- Uploaded Column

- Column Present
- Actual Data Type
- Data Type Result
- Issue

Validation Type values:

- Expected Column
- Extra Uploaded Column
- Schema Detection
- Schema File
- Schema Columns

Column Present values:

- Pass
- Fail
- Extra
- Unknown

Data Type Result values:

- Pass
- Fail
- Skipped
- Unknown

Typical usage

Run the script directly:

```
python CheckList\db_checklist.py
```

Analyze a DB1 file with automatic schema detection:

```
analyze_excel_file(r"D:\Dubai\Dubai_DB1.csv")
```

Analyze a DB2 file with automatic schema detection:

```
analyze_excel_file(r"D:\Dubai\Dubai_DB2.xlsx")
```

Analyze with an explicit schema file:

```
analyze_excel_file(
    r"D:\Dubai\Dubai_DB1.csv",
    schema_file=r"D:\Dubai Data\CheckList\DB1_Schema.xlsx"
)
```

Analyze with a custom report output name:

```
analyze_excel_file(
    r"D:\Dubai\Dubai_DB1.csv",
    output_file=r"D:\Dubai Data\CheckList\Dubai_DB1_custom_report.xlsx"
)
```

Important notes

The schema actual-column names must exactly match the uploaded file column names.
The validation is case-sensitive because pandas column matching is case-sensitive.

The checklist does not modify the uploaded DB1 or DB2 file. It only reads the file and writes a separate Excel analysis report.

The checklist report is intended to complement the DB1 and DB2 pipeline outputs. It checks structure, completeness, and data type compatibility, but it does not

validate business rules such as whether a value is logically correct for a Dubai project or transaction.